SHA256 加密

SHA256 加密
- 简介
  - SHA256 的特点
  - 应用场景
  - 代码解析
  - 代码要点说明

简介

SHA256 是密码学中常用的哈希算法，属于 SHA-2（Secure Hash Algorithm 2）家族。它能将任意长度的输入数据转换成固定长度（256位/32字节）的输出，这个输出通常以64位十六进制数表示。

SHA256 的特点

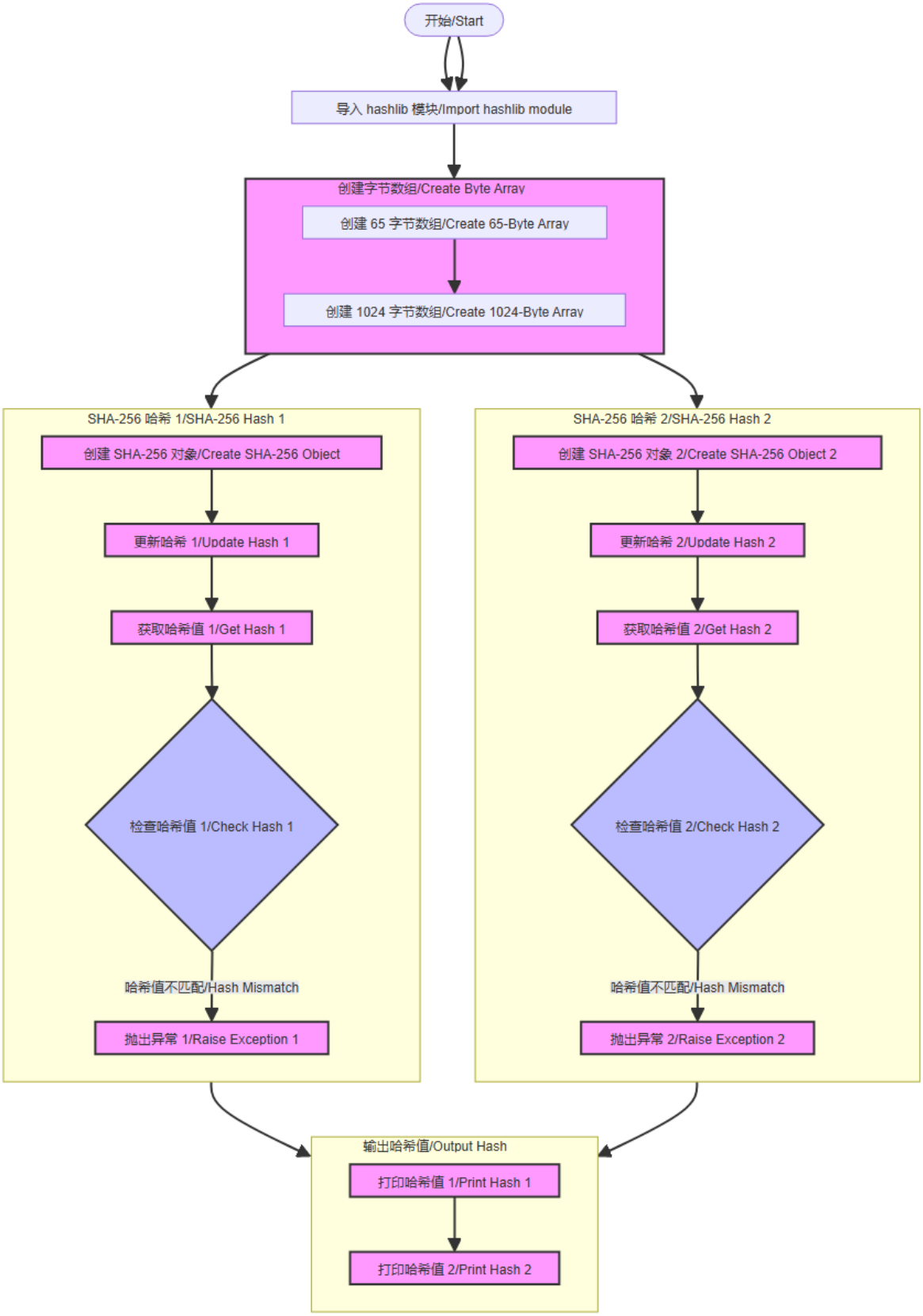
- **单向性**：从输出很难推导出输入
- **确定性**：相同输入总是产生相同输出
- **雪崩效应**：输入的微小变化会导致输出的显著不同
- **抗碰撞性**：找到两个不同输入产生相同输出非常困难
- **固定长度输出**：256位（32字节）

应用场景

1. 数字签名
2. 数据完整性验证
3. 密码存储
4. 区块链技术
5. 随机数生成

代码解析

例程代码执行流程图如下



完整版的代码位于 [源码汇总 / 02.Basic / 11.sha256.py]

```
# 导入 hashlib 模块，用于提供 SHA-256 哈希功能
# (Import the hashlib module for SHA-256 hashing functionality)
import hashlib

# 创建一个包含65个零字节的字节数组
# (Create a byte array containing 65 zero bytes)
```

```
a = bytes([0] * 65)

# 创建一个 SHA-256 哈希对象
# (Create a SHA-256 hash object)
b = hashlib.sha256()

# 更新哈希对象，使用字节数组 a 进行两次更新
# (Update the hash object with the byte array a twice)
b.update(a)
b.update(a)

# 获取最终的哈希值
# (Get the final hash value)
c = b.digest()

# 打印哈希值
# (Print the hash value)
print(c)

# 检查哈希值是否与预期值相同
# (Check if the hash value matches the expected value)
if c !=
b"\xe5Z\\'sj\x87a\xc8\xe9j\xce\xc0r\x10#%\xe0\x8c\xb2\xd0\xdb\xb4\xd4p,\xfe8\xf8
\xab\x07\t":
    # 如果不相同，则抛出异常并打印当前哈希值
    # (If not, raise an exception and print the current hash value)
    raise(Exception("error! {}".format(c)))

# 创建一个包含1024个零字节的字节数组
# (Create a byte array containing 1024 zero bytes)
a = bytes([0] * 1024)

# 使用字节数组 a 创建一个新的 SHA-256 哈希对象
# (Create a new SHA-256 hash object using the byte array a)
b = hashlib.sha256(a)

# 获取新的哈希值
# (Get the new hash value)
c = b.digest()

# 打印新的哈希值
# (Print the new hash value)
print(c)

# 检查新的哈希值是否与预期值相同
# (Check if the new hash value matches the expected value)
if c !=
b'_p\xbf\x18\xa0\x86\x00p\x16\xe9H\xb0J\xed;\x82\x10:6\xbe\xa4\x17U\xb6\xcd\xdf\
xaf\x10\xac\xe3\xc6\xef':
    # 如果不相同，则抛出异常并打印当前哈希值
    # (If not, raise an exception and print the current hash value)
    raise(Exception("error! {}".format(c)))
```

代码要点说明

1. **创建哈希对象的两种方式**：
   - 先创建空对象，然后用 `update()` 方法添加数据
   - 创建对象时直接传入数据
2. **增量哈希**：
   - 代码中第一部分展示了 `update()` 方法的使用，可以多次调用来添加数据
   - 这相当于计算连接在一起的数据的哈希值 (即 130 个零字节)
3. **哈希结果获取**：
   - `digest()` 方法返回二进制格式的哈希值
   - 也可以使用 `hexdigest()` 获取十六进制字符串表示
4. **哈希验证**：
   - 代码通过比较生成的哈希值与预期值来确保算法实现正确
   - 展示了SHA256算法的确定性特性（相同输入总是产生相同输出）
5. **输入大小与哈希值**：
   - 无论输入数据大小如何（65个字节、130个字节或1024个字节），输出始终是32字节

SHA256虽然强大，但在某些安全场景下（如密码存储）应该与其他技术如加盐、密钥拉伸等结合使用。